FORMATION « PYTHON EN 24 HEURES » | SÉANCE 11

Erreurs et exceptions : écrire du code robuste

Chapitre 20 du Tome 1

1 h de cours (présentation Beamer) • 1 h de pratique (notebook Jupyter)

HOUNKPE Mawulikplimi Roland et DJABON Ounimborbitibou

Objectifs de la séance

- Classer les erreurs en trois familles : syntaxe, exécution, logique
- Lire un traceback de bas en haut pour localiser la cause
- Protéger un traitement avec try, except, else et finally
- Lever des exceptions utiles avec raise et créer ses propres classes d'erreurs
- Adopter la philosophie EAFP face à LBYL

Rappel de la séance 10

Dans le mini projet, les validations lèvaient déjà ValueError et KeyError : cette séance les transforme en véritable système de gestion d'erreurs.

- Trois classes qui se parlent : Client, Compte, Banque
- Validations : jamais de solde négatif, jamais de virement douteux
- L'historique comme fil d'audit de chaque compte
- super() et composition pour assembler les objets
- Le rapport : assembler des __str__ en texte lisible

Trois familles d'erreurs

Erreurs de syntaxe

- Détectées AVANT l'exécution
- Deux points oublié, parenthèse non fermée
- Python refuse de démarrer

Erreurs d'exécution

- La syntaxe est bonne, le plantage vient en cours de route
- ZeroDivisionError, TypeError, KeyError
- Ce sont les **exceptions** de cette séance

Erreurs logiques

- Le programme tourne mais donne un faux résultat
- Python ne peut rien voir : seul le test détecte
- Chasse dédiée à la séance 12

Lire un traceback de bas en haut

```
def moyenne(notes):
    return sum(notes) / len(notes)

def rapport(notes):
    return f"Moyenne : {moyenne(notes):.2f}"

rapport([])
```

Le dernier ligne du traceback nomme l'exception : ZeroDivisionError. Les lignes au dessus remontent le chemin de l'appel : lisez les de bas en haut.

Les exceptions courantes

Exception	Cause typique	Exemple
NameError	nom jamais défini	variable mal orthographiée
TypeError	opération entre types incompatibles	"a" + 5
ValueError	type bon, valeur inacceptable	int("douze")
IndexError	index hors bornes	liste[10] sur 4 éléments
KeyError	clé absente du dict	dico["inconnue"]
ZeroDivisionError	division par zéro	x / 0
FileNotFoundError	fichier introuvable	open("nrf.txt")

Notez la hiérarchie : toutes héritent d'Exception. TypeError et ValueError se distinguent par la nature du problème : type ou valeur.

try / except : intercepter

```
try:
    nombre = int("douze")
    print("Conversion réussie")
except ValueError:
    print("Ce n'est pas un nombre valide")

print("Le programme continue normalement")
```

Le bloc try tente l'opération ; si l'exception annoncée survient, le bloc except la traite et l'exécution repart après le bloc : plus aucun plantage.

Plusieurs except et le mot clé as

- Un try peut annoncer plusieurs except, du plus spécifique au plus général
- ‘as e‘ récupère l’objet exception pour l’analyser
- Un except nu (sans type) attrape TOUT : à proscrire, il masque les vrais bugs
- Exception comme filet final, jamais en première ligne

```
try:
    valeur = int("3x") / 0
except ValueError as e:
    print("Valeur fausse :", e)
except ZeroDivisionError as e:
    print("Division interdite :", e)
except Exception as e:
    print("Autre incident :", type(e).__name__)
```

else et finally : le protocole complet

```
try:
    fichier = open("data.txt", "r")
except FileNotFoundError:
    print("Fichier absent")
else:
    print(fichier.read())
    fichier.close()
finally:
    print("Passage garanti, erreur ou non")
```

else s'exécute seulement si aucun incident ; finally s'exécute TOUJOURS : libération de ressource, message d'audit, nettoyage.

raise : lever à bon escient

```
def fixer_age(age):
    if not isinstance(age, int):
        raise TypeError("l'âge doit être un entier")
    if age < 0:
        raise ValueError("l'âge ne peut pas être négatif")
    return age

fixer_age(-3)
```

raise interrompt la fonction et remonte l'incident à l'appelant : les validations des séances précédentes étaient déjà des raise.

EAFP contre LBYL

- **LBYL** : Look Before You Leap. Tester avant d'agir : ‘if "cle" in d: ...‘
- **EAFP** : Easier to Ask Forgiveness than Permission. Agir puis réparer : ‘try: ... except KeyError: ...‘
- La culture Python privilégie EAFP : le cas normal lit sans test superflu
- EAFP évite les conditions de course et les doubles lectures
- Choisir selon la lisibilité : un try propre vaut trois tests imbriqués

Exceptions personnalisées

```
class SoldeInsuffisantError(Exception):
    """Levée quand un retrait dépasse le solde."""
    pass

def retirer(solde, montant):
    if montant > solde:
        raise SoldeInsuffisantError(
            f"retrait de {montant} pour un solde de {solde}")
    return solde - montant

try:
    retirer(100, 500)
except SoldeInsuffisantError as e:
    print("Refusé :", e)
```

Une classe qui hérite d'Exception suffit. Un nom métier parle plus fort que ValueError générique : le code appelant attrape exactement ce qu'il attend.

Attention

Le except nu masque les bugs

‘except:‘ sans type attrape aussi KeyboardInterrupt et les erreurs de syntaxe dynamique : votre programme continue dans un état incohérent. Attrapez toujours le type le plus précis possible, et terminez vos chaînes par ‘except Exception as e‘ explicitement loggué si vous devez vraiment tout couvrir.

À retenir

L'essentiel de la séance 11

Les exceptions transforment un plantage en message exploitable. try protège, except traite du plus spécifique au plus général, else confirme le succès, finally nettoie toujours. raise signale une règle métier bafouée ; vos propres classes d'Exception nomment ces règles. EAFP : agir, puis réparer, est l'idiome Python.

Travaux pratiques (1 h) : Exceptions et code robuste

Cinq exercices pour rendre vos programmes inplantables : interception, conversion sûre, exception métier, fichier protégé et validateur de données. Les cellules de vérification de cette séance SONT des exemples de try/except bien écrits : inspirez vous en.

- **Exercice 1 (Basique)** : Division sécurisée
- **Exercice 2 (Basique)** : Conversion robuste avec valeur de secours
- **Exercice 3 (Intermédiaire)** : Compte bancaire avec exception métier
- **Exercice 4 (Intermédiaire)** : Lecture de fichier robuste avec finally
- **Exercice 5 (Défi)** : Validateur de données CSV

Ouvrez le notebook « Seance_11_erreurs_et_exceptions.ipynb » pour commencer.

Pour la prochaine séance

Conseil de travail

Séance 12 : débogage et synthèse. Nous chasserons les erreurs logiques avec la méthode en quatre temps, print instrumenté, assert et breakpoint(). Relisez le tableau des exceptions de cette séance : le quiz final vous interrogera dessus.